

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра системного программирования ИСП РАН

Выпускная квалификационная работа бакалавра

Применение многопоточного кольцевого буфера для
организации высокоскоростного обмена
диагностической информацией в среде виртуализации

Автор:

Студент Б05-232 группы
Абубакиров Владимир Андреевич

Научный руководитель:

Кандидат физико-математических наук
Камкин Александр Сергеевич

Научный консультант:

Скородумов Дмитрий Павлович



Москва 2026

Аннотация

Применение многопоточного кольцевого буфера для организации высокоскоростного обмена диагностической информацией в среде виртуализации

Абубакиров Владимир Андреевич

Современные платформы виртуализации используют набор подсистем, обеспечивающий взаимодействие между гостевыми и хостовыми операционными системами. Для задач низкоуровневой отладки, профилирования и формирования отчётов об ошибках важен канал передачи диагностических данных из гостевой операционной системы на хост. В рамках работы был разработан такой механизм, основанный на MPSC-кольцевом буфере, размещённом в общей памяти.

Содержание

1	Введение	5
2	Обзор существующих решений	9
2.1	Последовательный порт и debugcon	9
2.2	virtio-console и virtio-serial	10
2.3	Организация кольцевых буферов	11
3	Исследовательская часть работы	13
3.1	Требования к каналу диагностики	13
3.2	Выбор общей памяти и ММIO-уведомления	13
3.3	Общая архитектура	14
3.4	Модель виртуального устройства	15
3.5	MPSC-кольцевой буфер	16
3.6	Проверка целостности и переполнения	19
3.7	Чтение сообщений на стороне хоста	21
4	Практическая часть: драйвер UEFI	22
4.1	Обзор фаз Platform Initialization	22
4.2	Фаза SEC (Security)	23
4.3	Фаза PEI (Pre-EFI Initialization)	24
4.3.1	PEI до инициализации ОЗУ (pre-memory)	24
4.3.2	PEI после инициализации ОЗУ (post-memory)	24
4.4	Фаза DXE (Driver Execution Environment)	25
4.5	Фаза TSL (Transient System Load)	26
4.6	Фаза Runtime	26
4.7	Перезагрузка	27
4.8	Отладочная библиотека	27
5	Практическая часть: драйвер Windows	28
5.1	Задача драйвера устройства логирования	28
5.2	Схема взаимодействия с драйвером	28
5.3	Модель разрабатываемого драйвера	29
5.4	Обнаружение регистров через ACPI	29
5.5	Интерфейс для обращения из пользовательского режима	30

5.6	Интерфейс для обращения из других драйверов	31
6	Оценка полученного решения	33
6.1	Функциональная оценка	33
6.2	Методика измерения производительности	34
6.3	Результаты измерений	34
6.4	Выводы по результатам измерений	36
7	Заключение	38

1 Введение

Развитие информационной инфраструктуры всё чаще приводит к применению технологий виртуализации: от облачных платформ и центров обработки данных до персональных рабочих станций. Виртуализация — группа технологий, основанных на преобразовании формата или параметров программных или сетевых запросов к компьютерным ресурсам с целью обеспечения независимости процессов обработки информации от программной или аппаратной платформы информационной системы [1]. Виртуальная машина — виртуальная вычислительная система, состоящая из виртуальных устройств обработки, хранения и передачи данных, а также, при необходимости, программного обеспечения и пользовательских данных [1].

Виртуальные машины используются для разработки, тестирования, обучения и развёртывания серверных платформ. Облачные провайдеры предоставляют пользователям вычислительные ресурсы без необходимости покупать собственное оборудование; пользователи могут масштабировать мощности под нагрузку и платить только за фактическое использование ресурсов. Отдельным фактором роста виртуализации является развитие машинного обучения: обучение и развёртывание моделей требуют большого числа CPU, GPU и NPU, доступ к которым часто предоставляется именно через виртуализированную инфраструктуру.

Развитие технологий виртуализации предъявляет повышенные требования к инструментам диагностики и отладки. Такие инструменты должны обеспечивать сбор данных из гостевых операционных систем с минимальным влиянием на их временные характеристики. В задачах логирования и профилирования важны низкая задержка, высокая пропускная способность и предсказуемое поведение при одновременной генерации сообщений несколькими компонентами гостевой системы.

Ещё одной особенностью современных виртуальных машин является наличие сопутствующего программного обеспечения: инструментов общего буфера обмена, перенаправления USB-устройств, синхронизации времени и

разрешения экрана. Эти сервисы и драйверы также генерируют диагностические события. Поэтому система сбора диагностики должна поддерживать единый хронологически упорядоченный поток сообщений от нескольких источников внутри гостя и от самого эмулятора.

Разработка подобной системы подразумевает создание драйвера в гостевой операционной системе и виртуального устройства на стороне эмулятора. Виртуальное устройство выступает в роли конечного приёмника сообщений: оно получает диагностические данные, сериализует их и выводит в файл на стороне хоста. Такой подход сохраняет прозрачность для гостевой системы, поскольку устройство обнаруживается как обычное аппаратное устройство через стандартные механизмы прошивки и операционной системы.

Основой предлагаемого механизма является общая память, разделяемая между гостевой операционной системой и гипервизором. В отличие от последовательной записи в порт ввода-вывода или эмулируемый ММЮ-регистр, такой канал позволяет передавать сообщение одним копированием в буфер и использовать отдельное уведомление только для сообщения о появлении данных. Синхронизация доступа к буферу осуществляется при помощи атомарных операций и разделения стадий резервирования, копирования и фиксации записи. Это уменьшает объём работы в критическом пути записи и делает стоимость вызова функции записи менее зависимой от планирования потоков гостевой системы.

Отдельной задачей является передача сообщений из среды прошивки. BIOS в широком смысле представляет собой интерфейс между программным обеспечением и аппаратными средствами [2]. В современных виртуальных машинах эту роль обычно выполняет UEFI (Unified Extensible Firmware Interface). Спецификация UEFI предоставляет программный интерфейс на основе модульной архитектуры, не зависящий от архитектуры ЦПУ, а также поддерживает загрузку по сети и механизм Secure Boot [3]. На ранних стадиях загрузки память может быть ещё не инициализирована, поэтому ди-

агностический канал должен работать как до появления общего буфера, так и после передачи управления операционной системе.

Объект исследования в данной работе — механизм передачи диагностической информации между гостевой операционной системой и эмулятором в среде виртуализации.

Предмет исследования — многопоточный MPSC-кольцевой буфер как средство передачи диагностической информации из нескольких источников гостевой системы на хост.

Целью дипломной работы является разработка и оценка комплекса программных компонентов для высокоскоростной передачи диагностической информации в среде виртуализации. В состав разрабатываемого решения входят виртуальное устройство для эмулятора Stratovirt [4], драйвер для среды UEFI и драйвер для операционной системы Windows.

Для достижения поставленной цели требуется выполнить следующие задачи:

1. проанализировать существующие способы передачи диагностической информации из гостевой системы на хост;
2. сравнить альтернативные организации кольцевых буферов и выбрать подходящую схему для передачи сообщений из нескольких источников;
3. разработать виртуальное устройство LogDev на стороне эмулятора;
4. реализовать поддержку записи диагностических сообщений на стадиях UEFI и после перехода к runtime-сервисам;
5. разработать драйвер Windows и программные интерфейсы для записи сообщений из пользовательского режима и режима ядра;
6. определить методику оценки полученного решения.

К функциональным требованиям относятся поддержка ранних стадий загрузки UEFI, возможность записи из нескольких компонентов гостевой системы, сохранение хронологического порядка сообщений и доступ к буферу

из пользовательского и ядерного режима. К нефункциональным требованиям относятся малая задержка записи, масштабируемость при росте числа потоков-писателей и ограничение накладных расходов на обращения к драйверу.

Методологическую основу работы составляют анализ технической документации Stratovirt, UEFI и Windows Driver Kit, изучение алгоритмов организации кольцевых буферов, проектирование виртуального устройства и оценка полученного решения.

2 Обзор существующих решений

В данной главе рассматриваются два класса альтернатив: каналы передачи диагностической информации из гостевой системы на хост и способы организации буфера внутри такого канала. Эти вопросы связаны, но не совпадают: например, `virtio-serial` является каналом передачи, а `eBPF ring buffer` — вариантом организации очереди событий.

2.1 Последовательный порт и `debugcon`

Наиболее простой способ вывести диагностическое сообщение из гостевой системы — использовать последовательный порт или отладочную консоль (`debugcon`) [5]. Такой канал доступен на ранних этапах загрузки и хорошо поддерживается прошивками: стандартная `DebugLib` в EDK2 как правило выводит сообщения именно в `serial`-порт.

Недостаток этого подхода связан с тем, что запись выполняется через операции ввода-вывода или эмулируемые MMIO-регистры. В виртуальной машине такое обращение обычно приводит к выходу из гостевого режима в гипервизор, обработке операции в эмуляторе и возврату управления гостю. Если сообщение передаётся побайтно, то эти накладные расходы возникают для каждого символа. Поэтому последовательный порт удобен как универсальный и простой механизм, но плохо подходит для интенсивного потока диагностических событий.

Существуют способы уменьшить накладные расходы. Например, доступ к портам ввода-вывода можно разрешить пользовательскому процессу через TSS и карту разрешения портов ввода-вывода, а сами сообщения можно складывать в дополнительный кольцевой буфер на стороне гостя. Однако в таком случае получается отдельный протокол поверх последовательного устройства. Для него нужно отдельно решать задачи обнаружения канала в UEFI и Windows, передачи метаданных о буфере, синхронизации нескольких писателей и объединения сообщений с логом эмулятора.

2.2 virtio-console и virtio-serial

Устройства семейства virtio предназначены для эффективного взаимодействия гостя и хоста в виртуализированной среде [6]. Для передачи текстовых данных могут использоваться virtio-console и virtio-serial [7]. Они применяют virtqueue и лучше масштабируются, чем побайтовая запись в последовательный порт.

Тем не менее для рассматриваемой задачи у virtio-устройств есть ограничения:

- полноценная работа virtio требует инициализации устройства и драйвера, поэтому канал неудобен для самых ранних стадий UEFI;
- запись диагностического сообщения проходит через общий стек virtio, что добавляет служебные структуры, уведомления и обработку очереди;
- пользовательское приложение не может безопасно и напрямую писать в virtqueue без участия драйвера и согласованного протокола доступа;
- интеграция сообщений гостевой ОС и сообщений самого эмулятора требует отдельной логики синхронизации и форматирования.

Таким образом, virtio-console и virtio-serial являются альтернативой для уже загруженной ОС, но хуже соответствуют требованиям раннего логирования, потенциального отображения буфера в пользовательское пространство и минимизации накладных расходов на запись.

Таблица 1: Сравнение каналов передачи

Канал	Преимущества	Ограничения
Serial/debugcon	Простота, доступность на ранней загрузке	Частые VM-exit при побайтовой записи, слабая масштабируемость
virtio-console/virtio-serial	Стандартный канал виртуализации, virtqueue вместо побайтового вывода	Требует инициализации драйвера, не даёт прямой записи из userspace в общий буфер
LogDev	Разделяемая память, ранний синхронный режим, отдельное уведомление о появлении данных	Требует собственного виртуального устройства и драйверов

2.3 Организация кольцевых буферов

После выбора канала на основе разделяемой памяти остаётся задача организации самого буфера. Для передачи потока сообщений можно использовать несколько типов очередей:

1. **SPSC-буфер** (Single Producer Single Consumer) прост и быстр, если источник сообщений ровно один. Его можно использовать и с несколькими писателями, добавив внешний глобальный мьютекс, но тогда конкурентная запись фактически сериализуется, а стоимость вызова записи начинает зависеть от владельца блокировки.
2. **МPMC-очередь** (Multiple Producer Multiple Consumer) поддерживает несколько писателей и читателей, но для данной задачи избыточна: читателем является только поток на стороне эмулятора.
3. **MPSC-буфер с блокировкой** соответствует числу участников протокола, но блокировка делает задержку записи зависимой от планирования потоков внутри гостя и от планирования виртуальных CPU на хосте.
4. **MPSC-буфер с атомарным резервированием** также соответствует модели «несколько писателей — один читатель». Он требует более сложного алгоритма резервирования и фиксации записи, но уменьшает объём синхронизации в критическом пути записи.

В Linux eBPF используется кольцевой буфер для передачи событий из программ eBPF в пользовательское пространство [8]. Он решает похожую задачу: несколько источников событий формируют сообщения, а пользовательский процесс считывает их из общей области памяти. Поэтому eBPF ring buffer рассматривается в работе как алгоритмическая альтернатива для организации очереди событий. Его ключевое отличие состоит в том, что этап резервирования места синхронизируется с помощью блокировки.

В данной работе выбран MPSC-буфер с атомарным резервированием, так как он соответствует модели взаимодействия «несколько писателей в гостевой системе — один читатель в эмуляторе» и позволяет отделить быструю запись сообщения от более дорогого уведомления виртуального устройства.

3 Исследовательская часть работы

В данной главе обосновывается архитектура разрабатываемого канала диагностики и описывается алгоритм кольцевого буфера.

3.1 Требования к каналу диагностики

Из постановки задачи и обзора альтернатив следуют следующие требования:

1. канал должен работать на ранних стадиях UEFI, когда оперативная память может быть ещё недоступна;
2. после инициализации памяти запись должна выполняться без побайтового выхода в эмулятор;
3. несколько источников в гостевой системе должны иметь возможность писать сообщения в общий поток;
4. пользовательский режим должен иметь путь записи с минимальными накладными расходами на обращения к драйверу;
5. эмулятор должен объединять сообщения гостевой системы со своими отладочными сообщениями в хронологическом порядке;
6. переполнение буфера должно быть явно обнаруживаемым и управляемым.

Эти требования приводят к двухрежимной схеме: синхронная запись через регистр виртуального устройства используется до появления общей памяти, а основной режим работы после инициализации памяти строится на кольцевом буфере.

3.2 Выбор общей памяти и ММЮ-уведомления

Один из возможных вариантов архитектуры — разместить кольцевой буфер в эмулируемой ММЮ-области устройства и позволить гостевой системе писать сообщения прямо в эту область. Для KVM-подобной виртуализации

такой вариант невыгоден: обращение к эмулируемой ММО обрабатывается не как обычная запись в RAM гостя, а как операция устройства. Такая запись может приводить к выходу в эмулятор и обработке доступа в пользовательском процессе Stratovirt. При интенсивном логировании это превращает каждую запись в дорогое взаимодействие с гипервизором.

Поэтому в предлагаемой схеме буфер размещается в RAM гостевой системы. Гость записывает сообщение обычными инструкциями работы с памятью, а виртуальное устройство получает только адрес буфера и уведомление о появлении данных. В результате дорогое обращение к ММО-регистру требуется не для каждого байта, а для события завершения записи или принудительного сброса буфера.

Такой выбор также важен для пользовательского режима. Если страницы буфера отображены в адресное пространство пользовательского процесса, приложение может формировать запись без дополнительного копирования в промежуточный буфер драйвера. Драйвер при этом остаётся ответственным за обнаружение устройства, отображение памяти и выдачу корректного интерфейса доступа.

3.3 Общая архитектура

Разработанный механизм включает три основных компонента:

1. виртуальное устройство LogDev на стороне Stratovirt;
2. драйвер и библиотеку логирования для среды UEFI;
3. драйвер Windows и интерфейсы для пользовательского режима и режима ядра.

Каналом взаимодействия между компонентами служит кольцевой буфер, расположенный в памяти гостевой системы. Виртуальное устройство хранит параметры канала, предоставляет ММО-регистры для синхронной записи, адреса буфера и уведомлений, а также выводит сообщения в лог эмулятора.

Общая архитектура механизма представлена на рисунке 1.

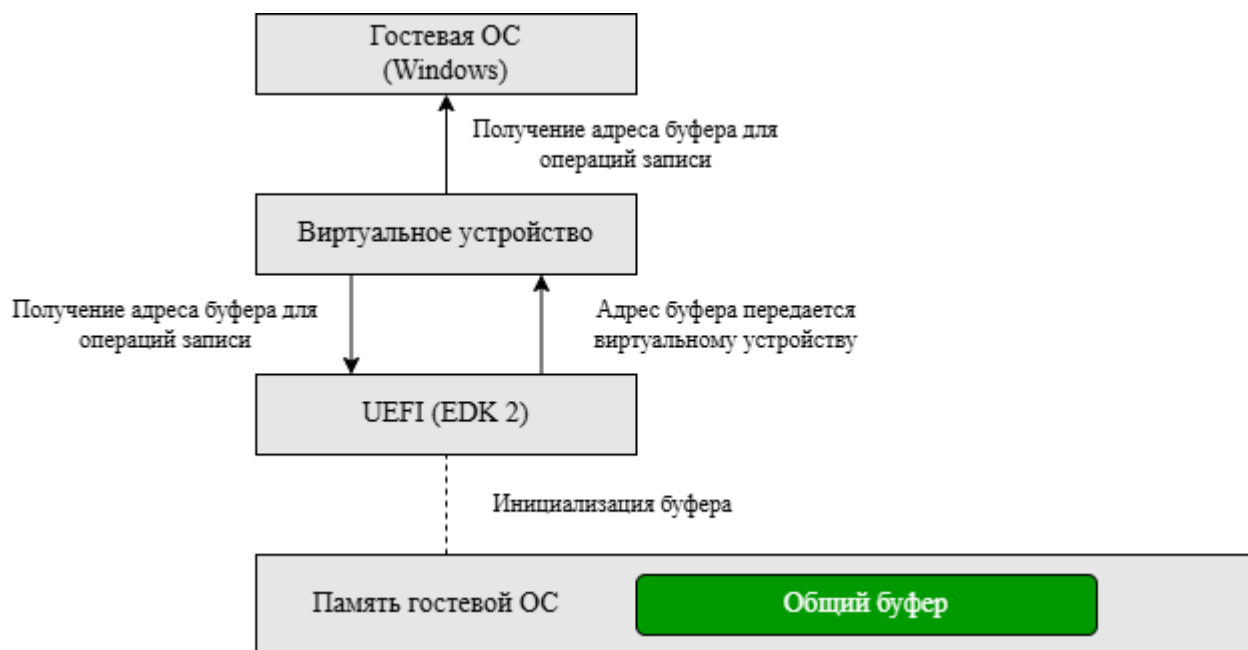


Рис. 1: Архитектура механизма обмена диагностической информацией

3.4 Модель виртуального устройства

Stratovirt — VMM (Virtual Machine Manager) экосистемы openEuler, ориентированный на облачные и контейнерные сценарии [4]. Как и QEMU, Stratovirt запускает виртуальную машину как отдельный процесс операционной системы. Внутри процесса выделяются главный поток обработки событий, потоки vCPU и, при необходимости, отдельные I/O-потоки (*iothreads*) [9].

Устройство *LogDev* присоединено к шине *SysBus* и занимает фиксированный диапазон в карте памяти гостя. Такое решение упрощает доступ к устройству из UEFI. Устройство добавляется в конфигурацию машины следующим параметром:

```

1 -device logdevice ,id=<logdev_id>[,flush_logs=<true|false>]
2 [,iothread=<iothread_id>]

```

Параметр *flush_logs* задаёт поведение при переполнении буфера: ожидать освобождения места или пропустить сообщение. Параметр *iothread* позволяет вынести чтение из буфера в отдельный поток, чтобы снизить нагрузку на главный цикл эмулятора.

Гостевая ОС взаимодействует с LogDev через ММIO-регистры, перечисленные в таблице 2.

Таблица 2: Регистры ММIO устройства LogDev

Смещ.	Имя	Доступ	Назначение
0x00	SYNC	запись	Синхронная передача до инициализации буфера
0x08	_WR BUFPTR	чтение/ запись	Физический адрес кольцевого буфера
0x10	KICK	запись	Уведомление эмулятора о наличии данных в буфере
0x18	OVERFLOW _BEHAVIOR	чтение	Текущее значение параметра <code>flush_logs</code>
0x20	FLUSH	запись	Принудительное чтение доступных сообщений
0x28	BUF SIZE	чтение	Размер полезной области буфера

Для обнаружения устройства операционной системой Stratovirt формирует запись в таблице ACPI: узел `LOGD` с идентификатором `_HID = STVRT_LOG` и ресурсом `_CRS`, описывающим диапазон ММIO-регистров устройства [10].

3.5 MPSC-кольцевой буфер

Общая память организована как кольцевой буфер с несколькими писателями и одним читателем (MPSC — Multiple Producer Single Consumer). В начале буфера располагается заголовок с позициями читателя и писателя, флагом переполнения и вспомогательной структурой `shadow_w_pos`. Эта структура хранит позицию резервирования для новых сообщений и число писателей, которые зарезервировали место, но ещё не завершили копирование.

```

1 typedef union {
2     struct {
3         uint32_t ReservedPos;
4         uint32_t WriterCnt;
5     } S;
6     uint64_t U;
7 } SHADOW_W_POS;
```



```
8
9 typedef struct {
10     uint32_t ReadPos;
11     uint32_t WritePos;
12     uint64_t IsOverflow;
13     SHADOW_W_POS ShWPos;
14     char Buffer[0];
15 } LOCKLESS_RING_BUFFER;
```

Запись выполняется в три этапа:

1. **reserve** — писатель атомарно резервирует уникальную область буфера;
2. **copy** — сообщение копируется в зарезервированную область;
3. **commit** — писатель сообщает о завершении копирования, после чего данные становятся доступными читателю.

На этапе резервирования поток считывает текущее значение `SHADOW_W_POS`, увеличивает счётчик активных писателей и сдвигает позицию резервирования на размер сообщения. Обновление выполняется через атомарную операцию `compare_exchange`; при конфликте попытка повторяется.

```
1 uint32_t LrbReserve (
2     LOCKLESS_RING_BUFFER *Rb,
3     const uint32_t MsgSize,
4     SHADOW_W_POS *Curr
5 ) {
6     for (;;) {
7         Curr->U = Rb->ShWPos.U;
8         SHADOW_W_POS Next = {
9             .S.ReservedPos = Curr->S.ReservedPos + MsgSize,
10            .S.WritePos = Curr->S.WriterCnt + 1
11        };
12
13        if (atomic_compare_exchange(&Rb->ShWPos, &Curr->U, Next.U)) {
14            // SHADOW_W_POS была обновлена в буфере
15            // Таким образом, гарантируется, что ни один другой поток
16            // не будет копировать своё сообщение в занятую память.
17            return Curr->S.ReservedPos;
18        }
19    }
```

```
19     }
20 }
```

После копирования сообщения писатель уменьшает счётчик активных писателей. Если после уменьшения счётчик становится равен нулю, значит все сообщения до текущей позиции уже скопированы, и глобальная позиция `WritePos` может быть продвинута. Читатель видит только область до `WritePos`, поэтому не считывает частично скопированные сообщения.

```
1 void LrbCommit (
2     LOCKLESS_RING_BUFFER *Rb,
3     SHADOW_W_POS *Curr
4 ) {
5     for (;;) {
6         SHADOW_W_POS NextShadowPos = {
7             .S.ReservedPos = Curr->S.ReservedPos,
8             .S.WritePos = Curr->S.WriterCnt - 1
9         };
10
11         if (NextShadowPos.S.WriterCnt == 0) {
12             // все сообщения до текущего закончили копирование
13             Rb->WritePos = Curr->S.ReservedPos;
14         }
15         if (atomic_compare_exchange(&Rb->ShWPos, &Curr->U, Next.U)) {
16             return;
17         }
18     }
19 }
```

Итоговая функция записи объединяет все три стадии. Сначала она проверяет возможность записи сообщения, затем резервирует место, копирует сообщение и фиксирует завершение копирования.

```
1 uint32_t LrbWrite(
2     LOCKLESS_RING_BUFFER *Rb,
3     const void *Msg,
4     uint32_t MsgSize
5 ) {
6     SHADOW_W_POS Curr;
7     CONST INT32 ReservedPos = LrbReserve(Rb, MsgSize, &Curr);
```

```
8
9 // Копирование сообщения в кольцевой буфер.
10 UINT32 WriteChunk = mRingBufferSize - ReservedPos;
11 if (WriteChunk > Size) {
12     WriteChunk = Size;
13 }
14 CopyMem(Rb->Buffer + ReservedPos, Msg, WriteChunk);
15 if (WriteChunk < Size) {
16     CopyMem(Rb->Buffer + ReservedPos, Msg + WriteChunk, Size - WriteChunk);
17 }
18
19 LrbCommit(Rb, &Curr);
20 }
```

3.6 Проверка целостности и переполнения

Перед резервированием места писатель должен убедиться, что в буфере достаточно свободного пространства. Для этого вычисляется размер занятой области как разность между позицией резервирования и позицией читателя. Простая проверка может ложно принять буфер за повреждённый, если между чтением `ShWPos` и чтением `ReadPos` другой писатель завершил запись, а читатель успел продвинуть позицию чтения.

Рассмотрим это на примере:

1. позиция читателя `ReadPos = 100;`
2. позиция писателя `WritePos = 200;`
3. количество активных писателей `ShWPos.WriterCnt = 1.`

При описанном ниже планировании потоков возникает указанная проблема.

Поток-писатель №1	Поток-писатель №2	Поток-читатель
(1) Операция резервирования: Curr->U = Rb->ShWPos.U (=> Curr->S.ReservedPos = 200) // Поток не увеличил ShWPos.WriterCnt // Другие потоки могут обновить WritePos		
	(2) Операция резервирования: ShWPos.U = Curr->U (=> ShWPos.S.ReservedPos = 300)	
	(3) Операция commit: WritePos = ShWPos.S.ReservedPos = 300 // WritePos смог измениться, т.к. // поток №1 не увеличил WriterCnt	
		Читает: ReadPos = WritePos = 300
Операция резервирования: uint32_t ReadPos = Rb->ReadPos = 300 uint32_t BusySize = Curr->S.ReservedPos - ReadPos = = 200 - 300		

Таблица 3: Пример планирования потоков

Таким образом, переменная `BusySize` переполняется, и буфер ложно считается повреждённым.

Поэтому при подозрении на неконсистентное состояние значения перечитываются. Если `shWPos` не изменилось, состояние действительно считается ошибочным. Если изменилось, проверка повторяется с актуальными значениями.

```

1 uint32_t LrbReserve (
2     LOCKLESS_RING_BUFFER *Rb,
3     const uint32_t MsgSize,
4     SHADOW_W_POS *Curr
5 ) {
6     for (;;) {
7         Curr->U = Rb->ShWPos.U;
8         uint32_t ReadPos = Rb->ReadPos;
9         uint32_t BusySize = Curr->S.ReservedPos - ReadPos;
10
11         while (BusySize > mRingBufferSize) {
12             CONST UINT64 CurrCpy = Curr->U;
13             MemoryFence();
14             Curr->U = Rb->ShWPos.U;

```

```
15         ReadPos = Rb->ReadPos;
16
17         if (Curr->U == CurrCpy) {
18             return -1;
19         }
20         BusySize = Curr->S.ReservedPos - ReadPos;
21     }
22     if (BusySize + Size > BUFFER_SIZE) {
23         // Сообщение не поместилось.
24         // В зависимости от выбранного поведения, сообщение либо
25         // пропускается, либо ожидает завершения записи других.
26     }
27
28     ...
29 }
30 }
```

При недостатке свободного места возможны два режима. В режиме пропуска сообщение отбрасывается, а флаг переполнения фиксируется в заголовке буфера. В режиме ожидания писатель инициирует чтение накопленных данных и повторяет попытку резервирования после освобождения места.

3.7 Чтение сообщений на стороне хоста

Алгоритм читателя проще алгоритма писателей, так как читатель один. После получения уведомления через `kick` или `flush` виртуальное устройство последовательно считывает сообщения от `ReadPos` до `WritePos`. После обработки сообщения позиция `ReadPos` продвигается, освобождая место для новых записей.

С точки зрения хоста писатели в буфере не являются доверенными: гостевая ОС контролирует содержимое памяти, поэтому эмулятор обязан проверять границы сообщений, размер буфера и состояние указателей. В рамках данной работы предполагается, что внутри гостевой ОС доступ к интерфейсам записи выдаётся доверенным компонентам, но это не отменяет проверок на стороне эмулятора.

4 Практическая часть: драйвер UEFI

Данная глава посвящена реализации поддержки LogDev в среде UEFI при помощи EDK2. Драйвер для виртуального устройства необходим для выделения памяти для кольцевого буфера, а также для поддержки записи сообщений в буфер из модулей прошивки.

UEFI (Unified Extensible Firmware Interface) — интерфейс между операционной системой и микропрограммами, управляющими низкоуровневыми функциями оборудования. Стандарт UEFI выделяет несколько основных стадий. Для разработки виртуального устройства для логирования наиболее важны нюансы работы с памятью на ранних этапах прошивки. Например, на стадии SEC и PEI оперативная память ещё недоступна: вместо неё используется кэш процессора. Это ограничивает нас в работе с общей памятью в традиционном понимании, поэтому было разработано два режима работы с устройством логирования:

- **синхронный ММЮ** — побайтовая запись в регистр устройства `sync_wr`;
- **запись через буфер в оперативной памяти гостевой ОС** — кольцевой буфер с уведомлением о записи через регистр виртуального устройства `kic`. Доступен после инициализации памяти и передачи адреса буфера через регистр виртуального устройства `bufptr`.

4.1 Обзор фаз Platform Initialization

Спецификация UEFI PI определяет стадии работы прошивки [11]:

- **SEC** — Security Phase;
- **PEI** — Pre-EFI Initialization;
- **DXE** — Driver Execution Environment;
- **TSL** — Transient System Load;
- **RT** — Runtime.

При переходе от одной фазы к другой изменяется:

- доступная память;
- таблицы сервисов;
- набор загружаемых модулей.

Так, на ранних этапах (SEC и PEI) контроллер памяти ещё не инициализирован, поэтому работа напрямую с ОЗУ недоступна. Также на разных стадиях доступны разные сервисы — API, которые UEFI предоставляет разработчику модуля. Каждая фаза также характеризуется набором загружаемых модулей — исполняемых компонентов прошивки.

Сводное сопоставление приведено в таблице 4.

Таблица 4: Фазы PI/UEFI и канал диагностики в разрабатываемом решении

Фаза	Память	Работа с устройством	Особенности разработки драйвера устройства
SEC	Кэш процессора	Синхронное через MMIO SYNC_WR	Для записи доступен только стек
PEI	Кэш процессора + ОЗУ	Синхронное через MMIO SYNC_WR	На данном этапе происходит инициализация RAM — после неё можно выделить память для буфера
DXE	Полная	Асинхронная через буфер	—
TSL	Полная	Асинхронная через буфер	—
RT	Полная + виртуальные адреса	Асинхронная через буфер	Подписка на <code>SetVirtualAddressMap</code> для сохранения виртуальных адресов буфера

4.2 Фаза SEC (Security)

Назначение. SEC — первая фаза прошивки после сброса CPU. Её основными задачами являются: обработка аппаратных исключений, инициализация временной памяти через кэш, передача управления следующей стадии — PEI [11].

Ограничения. На данном этапе нет таблиц с сервисами. Нельзя вызывать `AllocatePool`, `AllocateRuntimePool` для выделения памяти под буфер, так как ОЗУ ещё не инициализирована и используется временная память через кэш процессора.

Особенности разработки драйвера. На данной стадии происходят самые ранние сбои, которые трудно отладить без специальных инструментов. Из-за ограничений на работу с памятью для обмена сообщениями с устройством на этой стадии используется синхронная запись в специальный регистр устройства `sync_wr`. Драйвер записывает сообщение в регистр порциями по 1, 2, 4 или 8 байт, а эмулятор накапливает эти сообщения, пока не встретит символ окончания строки `\0` [12].

4.3 Фаза PEI (Pre-EFI Initialization)

Назначение. Стадия PEI Core последовательно запускает PEIM (PEI modules), публикующие PPI (PEI-to-PEI Interface), необходимые для предоставления интерфейса для обращений от других модулей. На данной стадии также происходит инициализация физической памяти, что позволяет на последующих этапах отойти от использования синхронного регистра `sync_wr` для записи сообщений.

4.3.1 PEI до инициализации ОЗУ (pre-memory)

Как и на стадии SEC, здесь доступна лишь временная память через кэш процессора, поэтому ограничения работы с виртуальным устройством остаются теми же:

- работа возможна лишь через **синхронный ММЮ**;
- выделение страниц под кольцевой буфер по-прежнему невозможна.

4.3.2 PEI после инициализации ОЗУ (post-memory)

После выставления PPI `MemoryDiscovered` PEIM получают доступ к методу `PeiServicesAllocatePages`, позволяющему выделить память в ОЗУ. Поэтому на

данном этапе производится резервирование памяти, которая представляет собой:

1. непрерывный диапазон страниц под заголовок кольцевого буфера и массив данных;
2. тип памяти — `runtime`, благодаря чему память может быть доступна после вызова `ExitBootServices` во время работы ОС (таких как Windows или Linux);

Также после выделения памяти адрес сообщается эмулятору через регистр `VFPT` виртуального устройства. На дальнейших этапах модули могут получить адрес буфера через этот регистр.

4.4 Фаза DXE (Driver Execution Environment)

Назначение. На данной стадии происходит загрузка драйверов устройств (например, USB, графических, сетевых и т.д.), предоставляется набор сервисов и протоколов (например, GOP (Graphics Output Protocol) для графики) и подготавливается среда для загрузчика ОС [13, 14].

Особенности разработки драйвера. DXE-драйвер (`DXE_DRIVER`) виртуального устройства выполняет:

1. **получение адреса буфера** — чтение регистра `VFPT` с адресом буфера;
2. **обработку переполнения** — чтение регистра `OVERFLOW_BEHAVIOR`, которое согласовано с параметром `flush_logs` строки запуска эмулятора, а также чтение регистра `VFBSIZE` с размером буфера;
3. **регистрацию событий** — подписка на событие смены виртуальных адресов `SetVirtualAddressMap` для поддержки работы на последующих стадиях;
4. **публикацию протокола записи** — публикация интерфейса для обращения к устройству `LogDev` из других модулей.

Обращение к устройству. Как правило, модули обращаются к виртуальному устройству не напрямую, а через специальную реализацию `DebugLib`,

которая обрабатывает сообщения и в конечном счёте обращается к драйверу для отправки сообщения. Драйвер же, в свою очередь, копирует сообщение в кольцевой буфер (раздел 3.5) и по окончании записи оповещает устройство при помощи регистра `kicr`.

4.5 Фаза TSL (Transient System Load)

Назначение. Данная стадия — переходный этап между прошивкой и ОС. TSL начинается, когда управление передаётся загрузчику операционной системы. На этом этапе всё ещё доступны Boot Services (до тех пор, пока не будет вызвана функция `ExitBootServices`).

На этой фазе также могут печататься важные для отладки сообщения. Для драйвера `LogDev` никаких дополнительных действий выполнять на этой фазе не нужно: буфер уже был зарезервирован, а его адрес можно получить из регистра `BUFPTR`.

4.6 Фаза Runtime

ExitBootServices. Загрузчик ОС в какой-то момент вызывает `ExitBootServices`. Все зарегистрированные обработчики события выхода из Boot Services выполняются до возврата из `ExitBootServices`. Как правило, в них происходит освобождение ресурсов, используемых модулями на предыдущих этапах. Как только завершается вызов `ExitBootServices`, начинается фаза Runtime.

SetVirtualAddressMap. Когда ОС переходит на виртуальные адреса, вызывается runtime service `SetVirtualAddressMap`, при исполнении которого вызываются ранее установленные обработчики. Обработчик `EVT_VIRTUAL_ADDRESS_CHANGE` обязан оттранслировать указатели runtime-кода UEFI в виртуальные согласно новой таблице [11]. Так поступает и обработчик `LogDev`, который обновляет свой адрес буфера на виртуальный. Без этого шага драйвер обращался бы к неверным адресам. Однако эмулятор может работать с физической памятью напрямую, поэтому сообщать ему об изменении адреса буфера нет необходимости.

4.7 Перезагрузка

Стоит также упомянуть, что происходит при перезагрузке операционной системы. После деинициализации ОС вызывает `UEFI Runtime Service ResetSystem`, который никогда не возвращает управление. В обработчике данной функции также могут печататься важные отладочные сообщения. Резонно в эмуляторе добавить слой безопасности и запретить перезапись адреса буфера в `WUPTR`. В таком случае, однако, нужно оставить возможность перезаписи адреса буфера, т.к. при повторной работе UEFI будет выделен новый буфер с иным адресом. Как правило, во многих эмуляторах (в том числе в `Stratovirt`) есть возможность подписаться на перезагрузку гостевой операционной системы и сбросить адрес буфера.

4.8 Отладочная библиотека

Помимо драйвера была разработана специальная реализация библиотеки `DebugLib`. В стандартной сборке EDK2 макросы `DEBUG(...)` и `ASSERT(...)` выводят сообщения, как правило, в последовательный (`serial`) порт. Разработанная реализация `DebugLib` перенаправляет эти вызовы в виртуальное устройство `LogDev` через UEFI-драйвер. Отладочные сообщения от модулей прошивки, уже использующих `DebugLib`, направляются в `LogDev` без необходимости изменять код существующих модулей.

5 Практическая часть: драйвер Windows

В данной главе рассматривается архитектура драйверов Windows и способы обращения пользовательских приложений и драйверов режима ядра к общему кольцевому буферу.

5.1 Задача драйвера устройства логирования

В гостевой ОС Windows диагностические сообщения могут быть сгенерированы:

- приложениями и сервисами пользовательского режима;
- другими драйверами режима ядра.

В рамках данной задачи подразумевается, что диагностические сообщения могут отправлять приложения, сервисы и драйверы, которым гостевая ОС выдала доступ к интерфейсу LogDev. Для хоста эти источники всё равно остаются недоверенными, поэтому границы сообщений и состояние буфера проверяются на стороне эмулятора.

Драйвер играет роль единой точки доступа к кольцевому буферу: он предоставляет интерфейс для записи сообщений в буфер, совершая проверки его целостности, и уведомляет эмулятор через ММЮ-регистр кикс (раздел 3.4).

5.2 Схема взаимодействия с драйвером

Модель Windows разделяет выполнение на пользовательский режим и режим ядра. Запросы к устройствам оформляются как IRP (I/O Request Packet), которые проходят стек драйверов, начиная с I/O Manager [15].

Схема взаимодействия с драйвером:

1. PnP Manager обнаруживает устройство через ACPI-таблицы;
2. функциональный драйвер создаёт Device Object (структура ядра Windows, предоставляющая интерфейс для I/O-взаимодействия с устройством);

3. пользовательский код открывает символьную ссылку на устройство (`\\.\\LogDev`); и вызывает `DeviceIoControl`;
4. драйвер обрабатывает IOCTL и обращается к разделяемой памяти.

5.3 Модель разрабатываемого драйвера

В рамках данной задачи драйвер был разработан по модели WDM (Windows Driver Model) в силу меньшего количества накладных расходов. Точкой входа в драйвер в данной модели является функция `DriverEntry`, инициализирующая структуру `DRIVER_OBJECT` и регистрирующая обработчики для различных типов запросов. `DriverUnload` является обработчиком для освобождения ресурсов, выделенных драйвером.

В `DriverEntry` регистрируются обработчики для различных типов запросов:

- `MajorFunction[IRP_MJ_CREATE]` — открытие устройства (вызывается при `CreateFile` или `OpenFile` в пользовательском режиме);
- `MajorFunction[IRP_MJ_CLOSE]` — закрытие (вызывается при `CloseHandle` в пользовательском режиме);
- `MajorFunction[IRP_MJ_DEVICE_CONTROL]` — обработка IOCTL (вызывается при `DeviceIoControl` с определённым кодом в пользовательском режиме);
- `MinorFunction[IRP_MN_START_DEVICE]` — запрос, который PnP менеджер отправляет драйверу для запуска устройства;
- `AddDevice` — данная функция вызывается PnP менеджером, когда устройство обнаруживается. Основной задачей данной функции является создание `Device Object` и прикрепление его к стеку устройств.

5.4 Обнаружение регистров через ACPI

Для большей универсальности адрес и размер ММIO-регистров передаются через ACPI-таблицы. Далее драйвер из регистров получает физический адрес и размер кольцевого буфера, выделенного в UEFI.

Напомним, что эмулятор добавляет в таблицы ACPI устройство со следующими параметрами [12]:

- имя ACPI-узла: LOGD;
- `_HID`: STVRT_LOG;
- `_CRS`: дескриптор `Memory32Fixed` для ММЮ-региона устройства LogDev (база и длина согласованы с картой памяти эмулятора).

Драйвер в обработчике `IRP_MN_START_DEVICE` получает структуру `IO_STACK_LOCATION` и извлекает из неё ресурсы устройства (дескриптор `CM_RESOURCE_LIST`). Из него извлекается диапазон ММЮ для регистров (таких как `BUFPTR`, `KICK` и др.). Далее драйвер получает физический адрес и размер кольцевого буфера и отображает физические страницы в виртуальное адресное пространство при помощи функций `MmMapIoSpace` / `MmGetPhysicalAddress` (в зависимости от типа памяти — ММЮ-регистры или RAM) и сохраняет указатель на структуру заголовка буфера.

5.5 Интерфейс для обращения из пользовательского режима

Для взаимодействия с буфером из приложений и сервисов пользовательского режима в драйвере есть отдельный IOCTL, принимающий готовое сообщение (NUL-терминированная строка). Обработчик, выставленный в `MajorFunction[IRP_MJ_DEVICE_CONTROL]`:

1. получает входной буфер и его тип при помощи `IoGetCurrentIrpStackLocation`;
2. проверяет размер (не превышает размер буфера);
3. вызывает общую функцию записи в MPSC-буфер (по реализации аналогична той, что в UEFI);
4. при успехе записывает в ММЮ `KICK` магическое значение `0xDEADBABA` для уведомления потока читателя.

Коды IOCTL задаются при помощи макроса `CTL_CODE` с уникальным для устройства значением `DeviceType`, кодом `Function`, параметром `Method` (в случае

данного драйвера поддерживается только `METHOD_NEITHER`) и флагом `Access`, определяющим права доступа [15].

Windows поддерживает 4 способа доступа к пользовательским данным:

1. `METHOD_BUFFERED` — при котором ядро создаёт промежуточный буфер, копирует в него пользовательский буфер и передаёт его драйверу;
2. `METHOD_IN_DIRECT` — I/O Manager проверяет пользовательские страницы, предотвращая их сброс на диск, и передаёт заполненную структуру MDL, позволяющую драйверу читать данные напрямую;
3. `METHOD_OUT_DIRECT` — то же, что `METHOD_IN_DIRECT`, но для записи из драйвера в пользовательский буфер;
4. `METHOD_NEITHER` — драйвер напрямую получает виртуальные адреса пользовательских буферов.

Для данной задачи был выбран именно последний метод передачи данных, так как он позволяет достичь максимальной производительности. Разработанный драйвер самостоятельно проверяет буферы на валидность при помощи функций `ProbeForRead` и `ProbeForWrite`.

5.6 Интерфейс для обращения из других драйверов

При получении сообщения от других драйверов режима ядра нет необходимости проверять буфер на валидность, поэтому был разработан второй ИЮСТЛ, доступный только из режима ядра, который возвращает указатель на функцию для записи:

```
1 typedef NTSTATUS (*LOGDEV_WRITE_FN)(  
2     PVOID Message ,  
3     ULONG MessageSize  
4 );
```

Таким образом, другой драйвер может работать с буфером напрямую, минуя дополнительные проверки адреса буфера. Более того, появляется возможность сэкономить на количестве вызовов драйвера.

Проверка того, из какого режима произошёл вызов осуществляется при помощи поля `RequestorMode` структуры `IRP`.

6 Оценка полученного решения

Оценка решения должна показать, что выбранная архитектура оправдана не только функционально, но и с точки зрения накладных расходов. Для этого сравниваются три варианта передачи диагностических сообщений: стандартный канал `virtio-serial`, LogDev с атомарным резервированием места в MPSC-буфере и LogDev с eBPF-подобной организацией MPSC-буфера.

6.1 Функциональная оценка

В таблице 5 приведены основные требования и способ их проверки в разработанном прототипе.

Таблица 5: Функциональная оценка решения

Требование	Статус и способ проверки
Логирование до инициализации RAM	Проверяется запуском UEFI с сообщениями на стадиях SEC/PEI
Переход к быстрому режиму после появления памяти	Проверяется появлением сообщений после выделения runtime-памяти и передачи адреса буфера устройству
Несколько источников сообщений	Проверяется записью из нескольких потоков и драйверов гостевой системы
Управляемое поведение при переполнении	Проверяется режимами пропуска сообщения и принудительного чтения накопленных данных
Доступ из пользовательского режима	Реализована запись через интерфейс Windows-драйвера.

Отдельно следует учитывать границы доверия. Компоненты, пишущие в буфер, могут быть доверенными с точки зрения гостевой ОС, но для хоста гостевая память остаётся недоверенной. Поэтому виртуальное устройство должно проверять размеры, позиции чтения и записи, а также корректно обрабатывать переполнение.

6.2 Методика измерения производительности

Для количественной оценки использовались два основных показателя. В виртуализированной среде основной интерес представляет стоимость вызова функции записи в гостевой системе: именно она влияет на задержки приложений и драйверов, генерирующих диагностические сообщения. Задержка доставки до хоста важна для скорости появления сообщения в итоговом логге, но она вторична для исполняемого кода внутри гостевой системы.

- **стоимость вызова записи** — время от входа в функцию записи в гостевой системе до возврата из неё;
- **задержка доставки** — время от отправки сообщения в гостевой системе до получения сообщения на стороне хоста.

Измерения проводились на хосте с процессором Kirin 9020 и 12 ГБ оперативной памяти. Гостевой системе было выделено 4 vCPU и 8 ГБ RAM. Размер кольцевого буфера составлял 0x10000 байт. В каждом эксперименте каждый поток отправлял по 500 сообщений.

Для обработки результатов использовались две отметки времени:

- **момент отправки** — время, когда сообщение было отправлено в гостевой системе;
- **момент получения** — время, когда сообщение было получено на стороне хоста.

6.3 Результаты измерений

В таблице 6 приведена задержка доставки сообщения от гостевой системы до хоста. Значения указаны в миллисекундах.

Таблица 6: Задержка доставки сообщения до хоста

Канал	Пот.	Среднее	СКО	Медиана
LogDev, MPSC атом. резервирование	1	338.27	3.67	337.32
LogDev, MPSC атом. резервирование	2	335.79	1.90	335.15
LogDev, MPSC атом. резервирование	4	335.78	1.94	335.23
LogDev, MPSC атом. резервирование	16	353.46	1608.28	335.11
LogDev, MPSC блокировка	1	798.85	0.55	798.82
LogDev, MPSC блокировка	2	804.51	4.32	803.98
LogDev, MPSC блокировка	4	803.24	3.67	802.91
LogDev, MPSC блокировка	16	799.34	1.89	799.19
virtio -serial	1	70.11	3.33	68.46
virtio -serial	2	69.99	2.78	68.84
virtio -serial	4	71.24	3.49	70.00
virtio -serial	16	70.07	3.10	68.60

В таблице 7 приведена стоимость вызова функции записи.

Таблица 7: Стоимость вызова функции записи

Канал	Пот.	Среднее	СКО	Вызовов/с
LogDev, MPSC атом. резервирование	1	0.036	0.640	8782
LogDev, MPSC атом. резервирование	2	0.068	0.544	17569
LogDev, MPSC атом. резервирование	4	0.099	0.685	20065
LogDev, MPSC атом. резервирование	16	0.521	1.452	12283
LogDev, MPSC блокировка	1	0.071	0.737	18225
LogDev, MPSC блокировка	2	0.053	0.538	20397
LogDev, MPSC блокировка	4	0.113	0.604	17835
LogDev, MPSC блокировка	16	0.527	1.277	11963
virtio -serial	1	0.216	0.961	3874
virtio -serial	2	0.249	0.961	3397
virtio -serial	4	0.192	1.053	4105
virtio -serial	16	0.228	0.914	3734

6.4 Выводы по результатам измерений

По результатам измерений можно сделать следующие выводы:

1. Стоимость вызова функции записи у LogDev ниже, чем у virtio-serial. По числу вызовов записи в секунду вариант LogDev с атомарным резервированием достигает 8.8–20.1 тыс. вызовов/с, а вариант с блокировкой — 12.0–20.4 тыс. вызовов/с. Для virtio-serial этот показатель находится в

диапазоне 3.4–4.1 тыс. вызовов/с.

2. При росте числа потоков LogDev сохраняет меньшую стоимость вызова записи в гостевой системе. Это важно для виртуализации, поскольку диагностический канал не должен заметно замедлять код, который генерирует сообщения.
3. По задержке доставки сообщения до хоста `virtio-serial` показывает меньшие значения: среднее значение находится около 70 мс, тогда как у LogDev с атомарным резервированием оно составляет около 336–338 мс при 1–4 потоках, а у варианта с блокировкой — около 799–805 мс. Эта метрика характеризует скорость появления сообщения в логе на хосте, но не стоимость записи для гостевого кода.

Таким образом, разработанный прототип уступает `virtio-serial` по задержке доставки до хоста, но снижает стоимость вызова записи в гостевой системе. Для диагностического канала в виртуализированной среде это является важным свойством, так как запись сообщения не должна существенно влиять на работу приложений, сервисов и драйверов, выполняющихся внутри гостя.

7 Заключение

В рамках работы был разработан механизм передачи диагностической информации из гостевой системы в эмулятор Stratovirt. Решение включает виртуальное устройство LogDev, драйвер для среды UEFI, адаптацию отладочной библиотеки EDK2 и драйвер для операционной системы Windows.

В первой части работы были рассмотрены альтернативные подходы к передаче диагностических сообщений: последовательный порт, `debugcon` и `virtio-console/virtio-serial`. Отдельно были рассмотрены способы организации кольцевого буфера, включая eVPF-подобный вариант с блокировкой при резервировании места. Анализ показал, что эти механизмы полезны в отдельных сценариях, но не совмещают все свойства, необходимые для рассматриваемой задачи: раннее логирование UEFI, запись из нескольких источников, единый лог гостя и эмулятора, а также возможность минимизировать накладные расходы при записи из пользовательского пространства.

Исследовательская часть работы обосновала выбор общей памяти и MPSC-кольцевого буфера. Разделение операций резервирования, копирования и фиксации записи позволяет нескольким писателям размещать сообщения в буфере и не открывать читателю частично скопированные данные. MMIO-регистры при этом используются не как основной канал передачи каждого байта, а как интерфейс настройки и уведомления виртуального устройства.

Практическая реализация поддерживает два режима работы. До инициализации оперативной памяти сообщения могут передаваться через синхронный регистр `sync_wr`. После выделения буфера в памяти гостевой системы основной поток сообщений передаётся через разделяемый кольцевой буфер, а эмулятор считывает данные после уведомления через `kick` или `flush`. Драйвер Windows предоставляет интерфейсы для записи из пользовательского режима и режима ядра.

Экспериментальная часть показала, что вызов функции записи в гостевой системе у LogDev дешевле, чем у `virtio-serial`: в проведённых измерениях

варианты LogDev обрабатывали до 20 тыс. вызовов записи в секунду, тогда как `virtio-serial` находился в диапазоне около 3.4–4.1 тыс. вызовов в секунду. По задержке доставки сообщения до хоста стандартный `virtio-serial` оказался быстрее реализованного прототипа: средняя задержка составила около 70 мс против примерно 336–338 мс у LogDev с атомарным резервированием места при 1–4 потоках.

Дальнейшее развитие решения может включать оптимизацию пути доставки сообщения на стороне хоста, поддержку дополнительных гостевых операционных систем, расширение модели безопасности для недоверенных писателей, добавление тайм-аутов для незавершённых записей и автоматизированный набор производительных тестов для регулярной проверки изменений в реализации.

Список литературы

- [1] *Росстандарт*. ГОСТ Р 56938-2016. Защита информации при использовании технологий виртуализации. Общие положения. — 2016.
- [2] *Вонг, А.* Справочник по параметрам BIOS / А. Вонг. — 2005. — DOI отсутствует.
- [3] «*Лаборатория Касперского*». UEFI. — Энциклопедия «Касперского», <https://encyclopedia.kaspersky.ru/glossary/uefi/>. — 2024. — Дата обращения: 15.06.2026.
- [4] *openEuler Community*. StratoVirt. — <https://www.openeuler.org/en/other/projects/stratovirt/>. — 2024. — Дата обращения: 15.06.2026.
- [5] *QEMU Project*. Debugging with QEMU debug console. — <https://www.qemu.org/docs/master/system/i386/pc.html>. — 2024. — Дата обращения: 15.06.2026.
- [6] *OASIS Open*. Virtual I/O Device (VIRTIO) Version 1.2. — <https://docs.oasis-open.org/virtio/virtio/v1.2/virtio-v1.2.html>. — 2022. — Дата обращения: 15.06.2026.
- [7] *Linux Kernel Documentation*. Virtio console. — <https://docs.kernel.org/driver-api/virtio/virtio.html>. — 2024. — Дата обращения: 15.06.2026.
- [8] *Linux Foundation*. BPF ring buffer. — <https://docs.kernel.org/bpf/ringbuf.html>. — 2024. — Дата обращения: 15.06.2026.
- [9] *openEuler Community*. StratoVirt Design. — <https://gitee.com/openeuler/stratovirt/blob/master/docs/design.md>. — 2024. — Дата обращения: 15.06.2026.
- [10] *UEFI Forum*. Advanced Configuration and Power Interface Specification. — <https://uefi.org/specifications>. — 2022. — Дата обращения: 15.06.2026.
- [11] *UEFI Forum*. UEFI Specification. — <https://uefi.org/specifications>. — 2022. — Дата обращения: 15.06.2026.

- [12] *Абубакиров, В. А.* StratoVirt with LogDev device (branch as_device_lockless). — https://gitee.com/abubakirovvladimir/stratovirt/tree/as_device_lockless/. — 2024. — Дата обращения: 15.06.2026.
- [13] *TianoCore.* EDK II. — <https://github.com/tianocore/edk2>. — 2024. — Дата обращения: 15.06.2026.
- [14] *TianoCore.* EDK II UEFI Driver Writer's Guide. — <https://tianocore-docs.github.io/edk2-UefiDriverWritersGuide/>. — 2020. — Дата обращения: 15.06.2026.
- [15] *Microsoft.* Windows Driver Kit documentation. — <https://learn.microsoft.com/en-us/windows-hardware/drivers/>. — 2024. — Дата обращения: 15.06.2026.
- [16] *QEMU Project.* QEMU Documentation. — <https://www.qemu.org/docs/master/>. — 2024. — Дата обращения: 15.06.2026.